

2026년 개인주제탐구발표대회 보고서

등교 시간 예측 프로그램

2026. 09. 04.

과 정 명	AI
소속 학교	대전가오중학교
성 명	박해진

대전교육정보원정보영재교육원

목차

I. 탐구(연구)의 필요성 및 목적

1. 탐구(연구)의 필요성
2. 연구 목적

II. 이론적 배경

1. 선행 연구
2. 용어 정의

III. 탐구(연구) 방법 및 과정(절차)

1. 연구 방법
2. 연구 과정

IV. 연구 결과

V. 결론 및 고찰

1. 결론
2. 고찰

※참고문헌

I. 탐구(연구)의 필요성 및 목적

1. 탐구(연구)의 필요성

등교를 매일 하다 보면 가끔씩 몇 시에 집에서 나가야 시간을 맞출 수 있는지 헷갈리곤 합니다. 너무 빨리 나가면 준비가 덜 된 채로 나갈 수도 있고, 너무 늦게 나가면 지각을 할 수도 있습니다. 항상 학교를 아슬아슬하게 시간에 맞춰 가는 저로서는 내가 몇 시에 나가야 하는지 항상 궁금했습니다. 또한 제 주변에도 이런 친구들이 꽤 많다는 걸 알고 나서는 등교 시간을 계산해 주는 프로그램이 필요함을 느꼈습니다.

2. 탐구(연구)의 목적

‘개인별로 최적화된 등교 시간을 알려주어, 지각을 최대한 줄이기’를 이 프로그램의 목적으로 삼았습니다.

II. 이론적 배경

1. 선행 연구

이제까지의 길찾기 앱 등 전통적인 최단 경로 탐색은 개인별 특성을 고려하지 않았기 때문에 개인의 속도를 예측하여 예상 시간을 출력하는 것에 무리가 있었습니다. 또한 기상 악화나 신호등 등을 정확하게 계산하지 못하기 때문에 예측 시간보다 더 빨리, 혹은 더 늦게 도착하는 일이 빈번했습니다.

2. 용어 정리

가. tkinter: python의 GUI를 표시하는 데 쓰이는 모듈입니다. 창을 만들 수 있고, 입력창이나 버튼을 표시하거나 messagebox 등을 표시할 수 있습니다.

나. sklearn: python에서 machine learning을 쉽게 할 수 있도록 만들어진 모듈. 쉽고 빠르게 모델을 생성할 수 있고, 분류, 회귀, 클러스터링 등을 할 수 있습니다.

다. pytorch(torch): python에서 신경망 래퍼, 레이어, 백엔드, 손실 함수, 옵티마이저, 스케줄러, 모델 컴파일, 데이터 엔지니어링 등을 할 수 있도록 만들어진 모듈입니다. 가장 널리 쓰이는 machine learning 모듈입니다.

III. 탐구(연구) 방법 및 과정(절차)

1. 연구 방법

python 설치 시 기본으로 제공되는 IDLE 에디터를 사용해 코딩하였습니다. (최신 버전인 3.14.7은 노트북의 문제로 pytorch가 제대로 작동하지 않아

GPU를 사용할 수 없어 3.12.10 버전을 사용하였습니다.)

machine learning은 sklearn 모듈과 pytorch 모듈을 사용하여 구현하였습니다. GUI는 tkinter 모듈을 사용하여 구현하였습니다.

pytorch에서 회귀 방식은 다층 퍼셉트론(MLP)을 이용한 회귀 방식을 사용하였습니다.

2. 연구 과정

가. 모듈 불러오기

```
1 import os
2 import threading
3 import pandas as pd
4 import numpy as np
5 import torch
6 import torch.nn as nn
7 from torch.utils.data import DataLoader, TensorDataset
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import mean_squared_error, r2_score
11 from tkinter import *
12 from tkinter import ttk, filedialog, messagebox, Toplevel
13 from PIL import Image, ImageTk
```

os, threading, pandas, numpy, torch, sklearn, tkinter, pillow(PIL) 모듈을 사용하였습니다.

나. 학습용 .csv 파일 가져오기

```
32 def load_file():
33     res = messagebox.askokcancel("파일 서식 확인", ".csv 파일은 서식에 맞춰 작성되어야 합니다.\n확인하시지 않으셨다면 도움말을 확인해주세요.")
34     if res == False:
35         return
36     global loaded_df
37     file_path = filedialog.askopenfilename(title = "테스트 데이터셋 선택", filetypes = [("CSV files", "*.csv")])
38
39     if file_path:
40         try:
41             if file_path.endswith('.csv'):
42                 loaded_df = pd.read_csv(file_path, encoding = 'cp949')
43                 file_label.config(text = f"파일 로드 완료: {file_path.split('/')[-1]}")
44             else:
45                 file_label.config(text = "파일이 .csv 형식이 아닙니다.")
46             return
47         except Exception as e:
48             file_label.config(text = "파일을 읽는 도중 오류가 발생했습니다.")
49             return
50         try:
51             loaded_df['weather'] = loaded_df['weather'].apply(lambda x: 1 if x == '비' else 0)
52             speed = (loaded_df['dist'] / 1000) / (((loaded_df['time'] - (loaded_df['weather'] * 1) - (loaded_df['traf'] * 1.25)) / 60))
53             speed_entry.insert(0, str(round(np.mean(speed), 1)))
54             window.after(0, lambda: messagebox.showinfo("자동 입력 안내", ".csv 파일에 입력된 데이터로 속도가 자동 계산되었습니다.\n변경할 수 있습니다."))
55         except KeyError as keye:
56             window.after(0, lambda: warning_label.config(text = "파일 내부 서식이 잘못되었습니다.\n확인하지 않으셨다면 도움말을 참고해 주세요."))
```

.csv 파일을 입력받고, 그 파일로 속도를 자동 계산합니다.

다. 데이터 준비 및 정리

```
201 global loaded_df
202 if loaded_df is not None and set(['dist', 'traf', 'weather', 'time']).issubset(loaded_df.columns):
203     df_eval = loaded_df.copy()
204     df_eval['weather'] = df_eval['weather'].apply(lambda x: 1 if x == '비' else 0)
205 else:
206     window.after(0, lambda: warning_label.config(text = "파일이 서식대로 작성되지 않았습니다.\n파일 작성법은 메뉴의 '도움말'을 참고해 주세요.", fg = "red"))
207     return
208 df_eval['pace'] = (df_eval['dist'] / 1000.0) / df_eval['dist'] / (df_eval['time'] - (df_eval['weather'] * np.random.normal(1, 1.5))
209     | (df_eval['traf'] * np.random.normal(1, 1.25, size = traf_input).sum()))
210
211 X_features = df_eval[['pace', 'traf', 'weather']].values
212 y_targets = df_eval[['time']].values
213 X_train, X_test, y_train, y_test = train_test_split(X_features, y_targets, test_size = 0.2, random_state = 42)
214
```

학습할 데이터를 정리합니다.

라. 입력창 초기화

```
dist_entry.delete(0, END)
traf_entry.delete(0, END)
fl_entry.delete(0, END)
speed_entry.delete(0, END)
epoch_entry.delete(0, END)
weather_combobox.current(0)
file_label.config(text = "")
warning_label.config(text = "")
result_label.config(text = "")
metrics_label.config(text = "")
```

마. 모델 학습시키기

```
210 X_features = df_eval[['pace', 'traf', 'weather']].values
211 y_targets = df_eval[['time']].values
212 X_train, X_test, y_train, y_test = train_test_split(X_features, y_targets, test_size = 0.2, random_state = 42)
213
214 scaler_X = StandardScaler()
215 scaler_y = StandardScaler()
216
217 X_train_scaled = scaler_X.fit_transform(X_train)
218 y_train_scaled = scaler_y.fit_transform(y_train)
219
220 X_test_scaled = scaler_X.transform(X_test)
221
222 X_train_tensor = torch.tensor(X_train_scaled, dtype = torch.float32).to(device)
223 y_train_tensor = torch.tensor(y_train_scaled, dtype = torch.float32).to(device)
224
225 X_test_tensor = torch.tensor(X_test_scaled, dtype = torch.float32).to(device)
226
227 dataset = TensorDataset(X_train_tensor, y_train_tensor)
228 loader = DataLoader(dataset, batch_size = 32, shuffle = True)
229 model = RegressionNet(input_size = X_features.shape[1]).to(device)
230 criterion = nn.MSELoss()
231 optimizer = torch.optim.Adam(model.parameters(), lr = 0.01)
232
233 model.train()
234 for epoch in range(epochs):
235     for batch_x, batch_y in loader:
236         optimizer.zero_grad()
237         output = model(batch_x)
238         loss = criterion(output, batch_y)
239         loss.backward()
240         optimizer.step()
241
242 if (epoch + 1) % 10 == 0 or epoch == epochs - 1:
243     progress = (epoch + 1) / epochs * 100
```



```

244         window.after(0, lambda perc = progress: update_label.config(text = f"학습 중... {perc:.2f}%"))
245
246     window.after(0, lambda: update_label.config(text = "학습 완료!"))
247
248     model.eval()
249     with torch.no_grad():
250         preds_scaled = model(X_test_tensor).cpu().numpy()
251
252     preds = scaler_y.inverse_transform(preds_scaled)
253     window.after(0, lambda: result_label.config(text = f"예상 소요 시간: 약 {avg_time:.1f} 분"))
254
255 except Exception as e:
256     window.after(0, lambda: warning_label.config(text = f"에러 발생!", fg = "red"))
257 finally:
258     window.after(0, lambda: button.config(state = "normal"))

```

바. GUI 창

```

273 weather_val = ["맑음", "비"]
274
275 label_title = Label(window, text = "등교 시간 예측기", font = ("맑은 고딕", 15, "bold"))
276 label_title.pack(pady = 10)
277
278 grid_frame = Frame(window)
279 grid_frame.pack(pady = 5)
280
281 Label(grid_frame, text = "등교 거리 (m)").grid(row = 0, column = 0, padx = 5, pady = 5)
282 dist_entry = Entry(grid_frame)
283 dist_entry.grid(row = 0, column = 1, padx = 5, pady = 5)
284
285 Label(grid_frame, text = "신호등 개수").grid(row = 1, column = 0, padx = 5, pady = 5)
286 traf_entry = Entry(grid_frame)
287 traf_entry.grid(row = 1, column = 1, padx = 5, pady = 5)
288
289 Label(grid_frame, text = "현재 날씨").grid(row = 2, column = 0, padx = 5, pady = 5)
290 weather_combobox = ttk.Combobox(grid_frame, values = weather_val, state = "readonly", width = 12)
291 weather_combobox.current(0)
292 weather_combobox.grid(row = 2, column = 1, padx = 5, pady = 5)
293
294 Label(grid_frame, text = "학습 횟수 (Epochs)").grid(row = 7, column = 0, padx = 5, pady = 5)
295 epoch_entry = Entry(grid_frame)
296 epoch_entry.grid(row = 7, column = 1, padx = 5, pady = 5)
297
298 Label(grid_frame, text = "이동 속도 (km/h)").grid(row = 8, column = 0, padx = 5, pady = 5)
299 speed_entry = Entry(grid_frame)
300 speed_entry.grid(row = 8, column = 1, padx = 5, pady = 5)
301
302 Label(grid_frame, text = "거주 층수").grid(row = 3, column = 0, padx = 5, pady = 5)
303 fl_entry = Entry(grid_frame)
304 fl_entry.grid(row = 3, column = 1, padx = 5, pady = 5)
305
306 Label(grid_frame, text = "-" * 50, font = ("맑은 고딕", 10), fg = "gray").grid(row = 6, column = 0, padx = 5, pady = 5, colspan = 2)
307
308 file_button = Button(grid_frame, text = "데이터 파일(.csv) 불러오기", command = load_file, bg = "#333333", fg = "white")
309 file_button.grid(row = 4, column = 0, padx = 5, pady = 5, colspan = 2)
310
311 file_label = Label(grid_frame, text = "파일 미선택", font = ("맑은 고딕", 8), fg = "gray")
312 file_label.grid(row = 5, column = 0, padx = 5, pady = 5, colspan = 2)
313
314 button = Button(window, text = "시작", command = start_learning_thread, width = 18, bg = "#007300", fg = "white", font = ("맑은 고딕", 10, "bold"))
315 button.pack(pady = 10)
316
317 update_label = Label(window, text = "", font = ("D2Coding", 9))
318 update_label.pack(pady = 2)
319
320 warning_label = Label(window, text = "", font = ("맑은 고딕", 9))
321 warning_label.pack(pady = 2)
322
323 result_label = Label(window, text = "조건 선택 후 시작 버튼을 누르세요.", font = ("맑은 고딕", 11, "bold"), fg = "#333333")
324 result_label.pack(pady = 5)
325
326 metrics_label = Label(window, text = "", font = ("맑은 고딕", 9, "bold"), fg = "#00008B")
327 metrics_label.pack(pady = 5)
328
329 window.mainloop()

```

사용자가 쉽게 사용할 수 있도록 GUI 창을 만듭니다.

사. 메뉴

1) 메뉴바 설정

```
266 menubar = Menu(window)
267 window.config(menu = menubar)
268 file_menu = Menu(menubar, tearoff = 0)
269 file_menu.add_command(label = ".csv 파일 입력 도움말", command = csv_help)
270 file_menu.add_command(label = "이동 속도 입력 도움말", command = speed_help)
271 menubar.add_cascade(label = "도움말", menu = file_menu, underline = 0)
272
```

2) csv 도움말

```
58 def csv_help():
59     current_dir = os.path.dirname(os.path.abspath(__file__))
60     img_path = os.path.join(current_dir, "csv_help.png")
61
62     if not os.path.exists(img_path):
63         alt_path = os.path.join(current_dir, "csv_help")
64         if os.path.exists(alt_path):
65             img_path = alt_path
66         else:
67             messagebox.showerror("파일 오류", f"도움말 이미지를 찾을 수 없습니다.")
68             return
69
70     try:
71         messagebox.showinfo("예시 데이터", ".csv 파일 입력 예시 데이터입니다.")
72         help_window = Toplevel(window)
73         help_window.title("예시 데이터")
74
75         raw_img = Image.open(img_path)
76
77         max_width = 500
78         w_percent = (max_width / float(raw_img.size[0]))
79         h_size = int((float(raw_img.size[1]) * float(w_percent)))
80
81         resized_img = raw_img.resize((max_width, h_size), Image.Resampling.LANCZOS)
82         tk_img = ImageTk.PhotoImage(resized_img)
83
84         help_window.geometry(f"{max_width + 20}x{h_size + 20}")
85
86         img_label = Label(help_window, image = tk_img)
87         img_label.image = tk_img
88         img_label.pack(padx = 10, pady = 10)
89
90     except Exception as e:
91         messagebox.showerror("오류", f"이미지를 표시하는 도중 오류가 발생했습니다")
```

3) 속도 입력 도움말

```
93 def speed_help():
94     messagebox.showinfo
95     ("""속도 입력 방식", "속도 칸은 .csv 파일을 입력하면 자동으로 채워집니다.
96         자신의 이동 속도를 알고 있다면 직접 입력할 수도 있습니다.""")
```

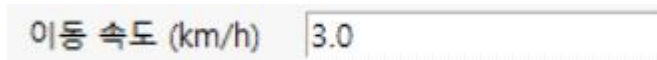
이 프로그램을 처음 이용하는 사람이 대략적인 이용 방법을 알 수 있게 메뉴에 도움말을 넣었습니다.

IV. 연구 결과

프로그램을 실행하게 되면 작은 GUI 창이 나옵니다.

‘데이터 파일 불러오기’ 버튼을 누르면 안내창이 나옵니다.

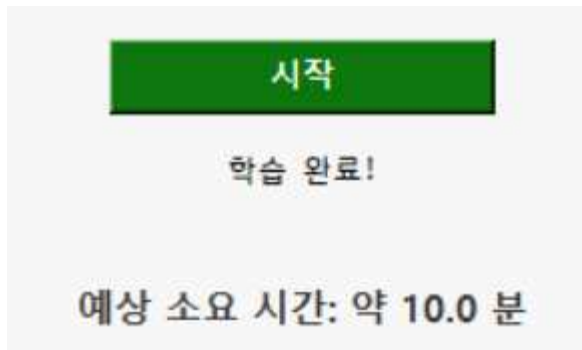
데이터 파일을 불러오면 속도가 자동 입력됩니다.



정보를 입력하고 시작 버튼을 누르면 학습이 시작됩니다.



학습이 완료되면 결과와 함께 시작 버튼이 다시 활성화됩니다.



메뉴에 ‘도움말’을 누르면 도움말을 볼 수 있습니다.

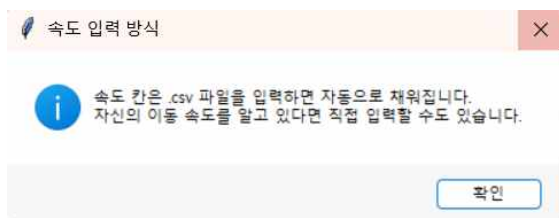
가. .csv 파일 입력 도움말

안내 문구를 띄운 후 예시 데이터가 나옵니다.



나. 이동 속도 입력 도움말

안내 상자가 나옵니다.



V. 결론 및 고찰

1. 결론

‘등교 시간 예상’이라는 변수가 많은 주제로 프로그램을 만들다 보니, 제가 예상하지 못한 곳에서 오류들이 터지기도 했습니다. 하지만 그런 오류들을 고치면서 나아가다 보니, 더 성장하고 발전한 저를 발견할 수 있었던 것 같습니다.

2. 고찰

분명히 제가 고치지 못한 오류나 변수가 있을 것입니다. 제가 이 프로그램을 혼자서 고칠 수 있는 수준이 된다면, 이 프로그램의 오류를 고치고 더 발전시켜 상용화하는 것을 목표로 더 나아가보고 싶습니다.

※참고문헌

tkinter messagebox: <https://m.blog.naver.com/spreatics2/222520827099>

sklearn MLP: <https://sjh9708.tistory.com/225>

pytorch 공식 튜토리얼: https://tutorials.pytorch.kr/beginner/nn_tutorial.html

pytorch 설치 방법: <https://blog.naver.com/rube2001/223914751224>

pytorch 공식 버전별 pytorch: <https://pytorch.kr/get-started/compatibility/>